

Lecture 13:
Pre-training and Fine-tuning Transformers

Instructor: Xiang Ren

USC CSCI 544 Applied NLP

Fall 2026

Some slides adapted from Dan Jurafsky and Chris Manning

Announcements

- CARC access activated – will have update by end of the week.

- HW1 due today – grading in two weeks.
- HW2 will be released on Thu.

- Tue 10/20: Project midterm presentation
  - TA will announce logistics for schedule sign-up
  - Rubric will be shared this week

- 10/27: Project midterm report due!

Mid-term Presentation

The project midterm presentation (2min presentation with 2 slides + 1min FAQ) will be given by your team in class and graded on clarity, novelty, and presentation quality (by instructor and TA). → rubric will follow!

1 member from the team can do the presentation representing the whole team.

Your presentation should convey:

Slide 1: An overview of the problem and your method (and differentiation with prior work)

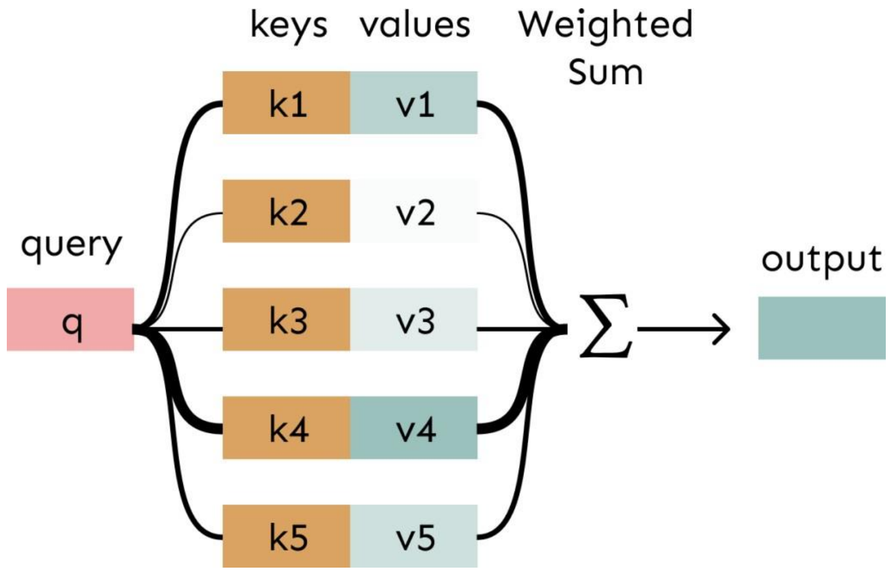
Slide 2: the main takeaway you have reached with your project so far; and an overview of what remains to be done before the final report.

Lecture Outline

- Announcements
- Recap: Transformers as Encoders, Decoders, Encoder-Decoders
- The pre-training and fine-tuning paradigm
  - Pre-training Decoder-Only Models
  - Pre-training Encoder-Only Models
  - Pre-training Encoder-Decoder Models
- Tokenization

Self-Attention

Keys, Queries, Values from the same sequence



- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- Step 2: Calculate attention score between **query** **and** **keys.**

$$e_{ij} = q_i \cdot k_j$$

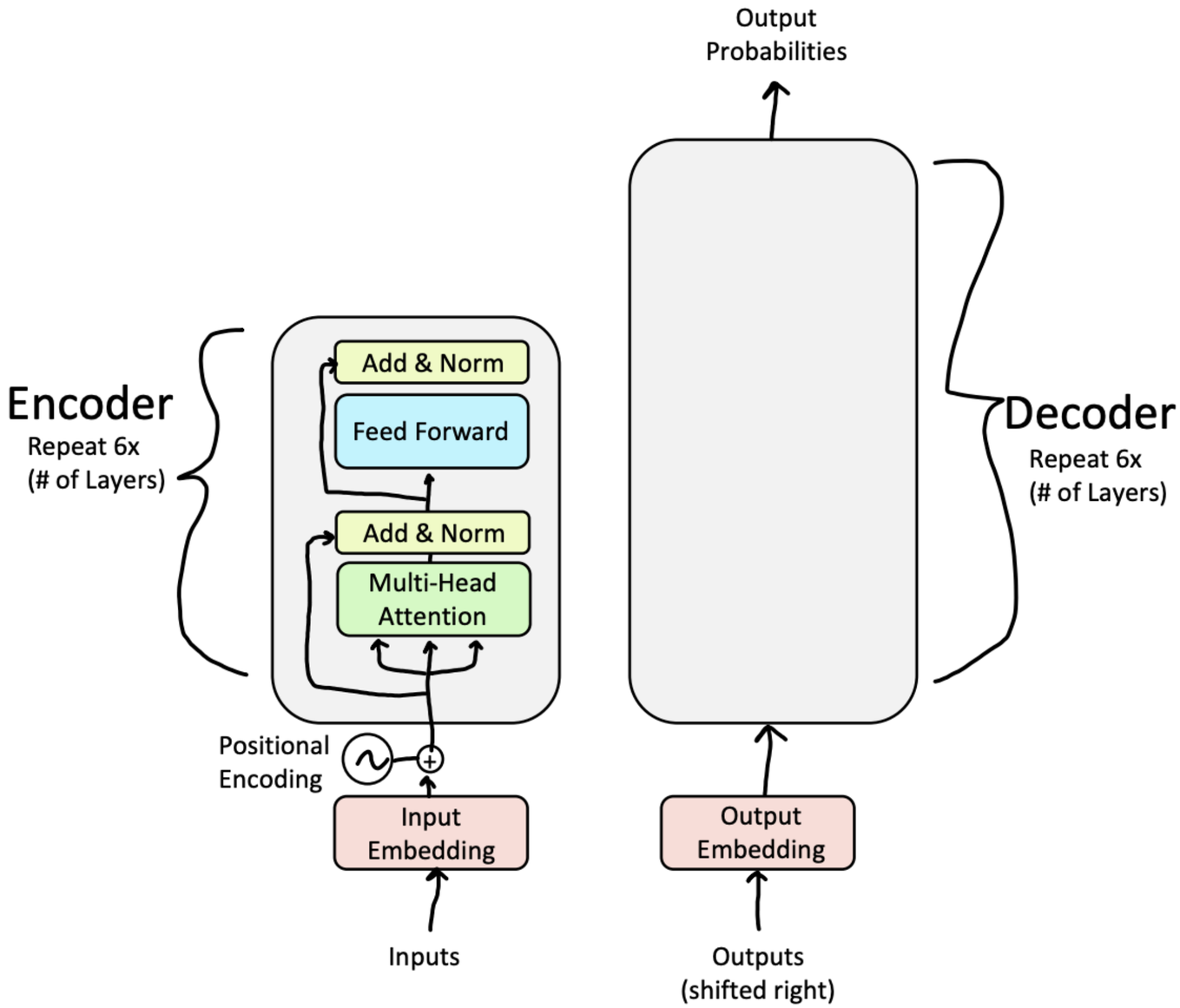
- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

- Step 4: Take a weighted sum of **values**.

$$Output_i = \sum_j \alpha_{ij} v_j$$

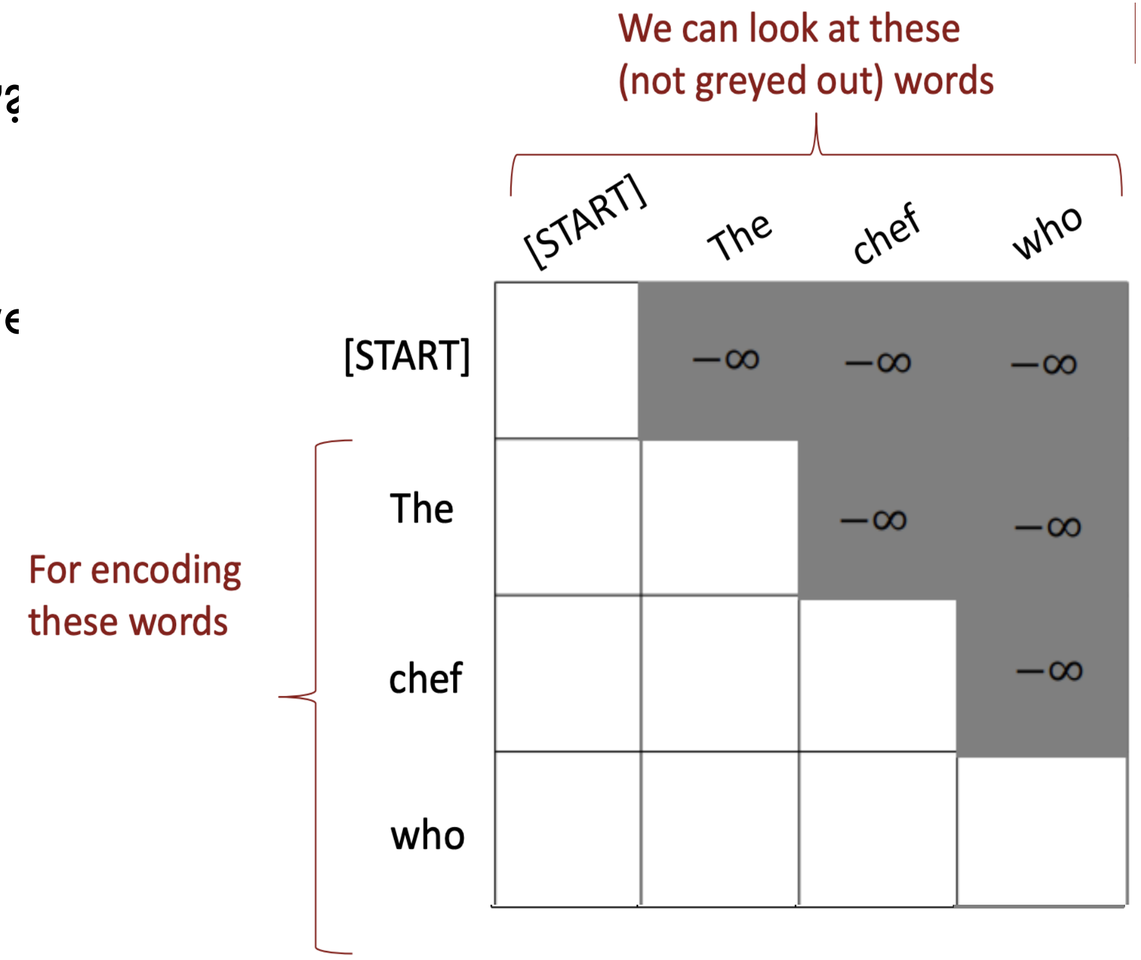
Yay, we've completed the Encoder! Time for Decoder ...



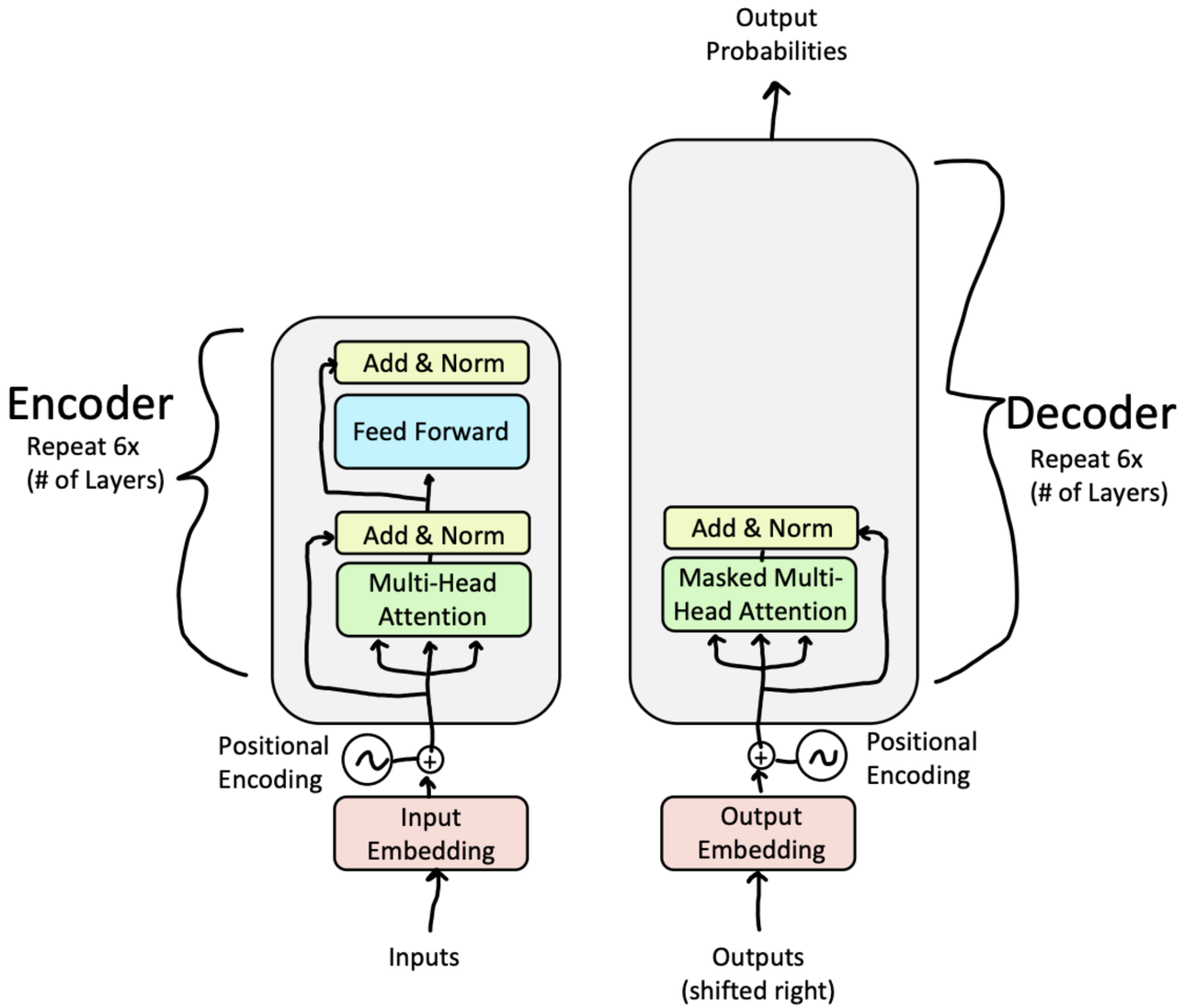
Decoder: Masked Multi-Head Self-Attention

- Problem: How do we keep the decoder from "cheating"?
  — looking into the future
- Solution: Masked Multi-Head Attention.
- To enable parallelization, we mask out attention to future words by setting attention scores to -∞

$$e_{ij} = \begin{cases} q_i^\top k_j, j < i \\ -\infty, j \geq i \end{cases}$$

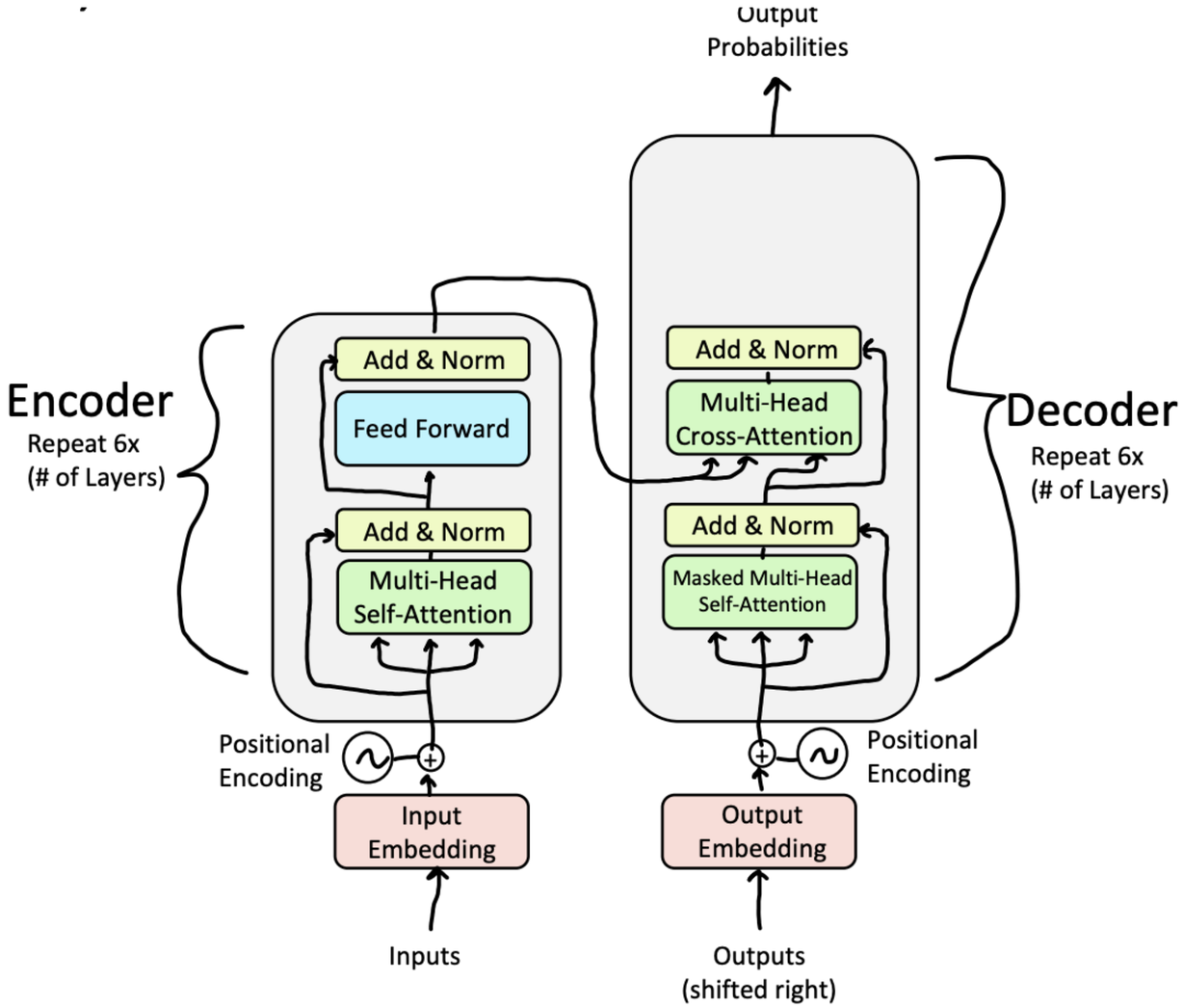


Decoder: Masked Multi-Head Self-Attention



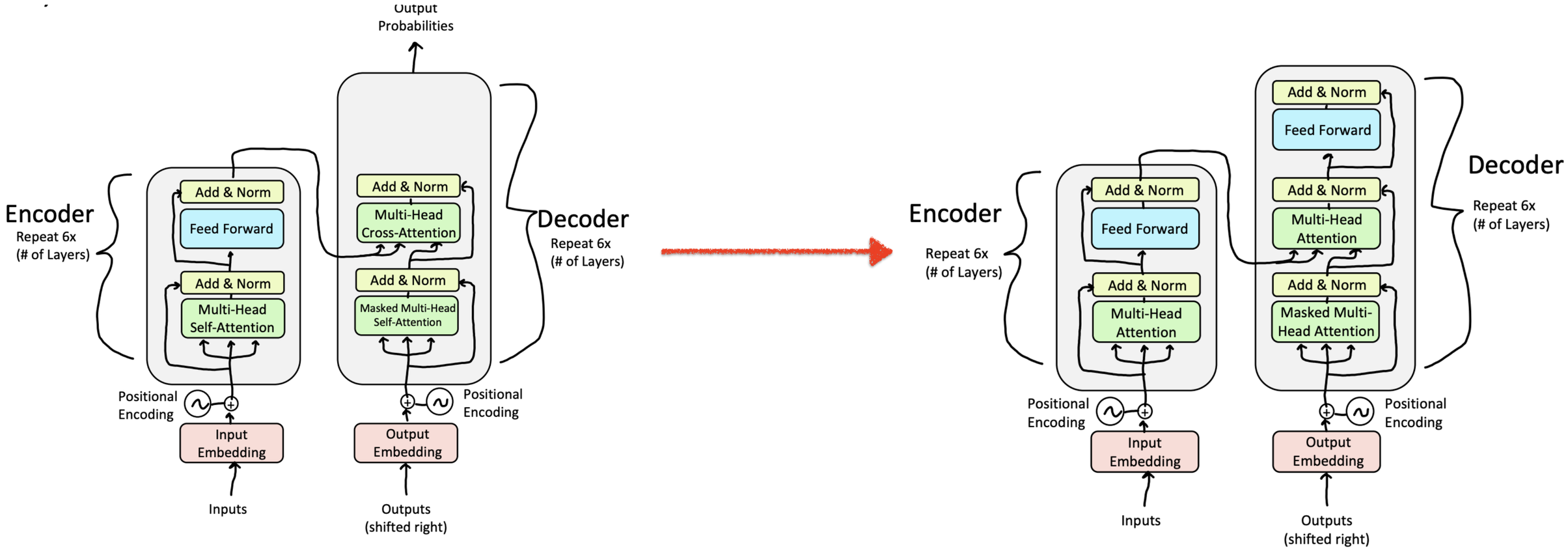
Encoder-Decoder Attention (Cross Attention)

- We saw that self -attention is when keys, queries, and values come from the same source.
- In the decoder, we have attention that looks more like what we saw last week.
- Let $\mathbf{h}_1, \ldots, \mathbf{h}_n$ be output vectors from the Transformer encoder, $\mathbf{h}_i \in \mathbb{R}^d$
- Let $\mathbf{z}_1, \ldots, \mathbf{z}_n$ be input vectors from the Transformer decoder, $\mathbf{z}_i \in \mathbb{R}^d$
- Then keys and values are drawn from the encoder (like a memory):
  - $\mathbf{k}_i = \mathbf{K}\mathbf{h}_i, \ \mathbf{v}_i = \mathbf{V}\mathbf{h}_i$
- And the queries are drawn from the decoder, $\mathbf{q}_i = \mathbf{Q}\mathbf{z}_i$



Decoder: Finishing touches!

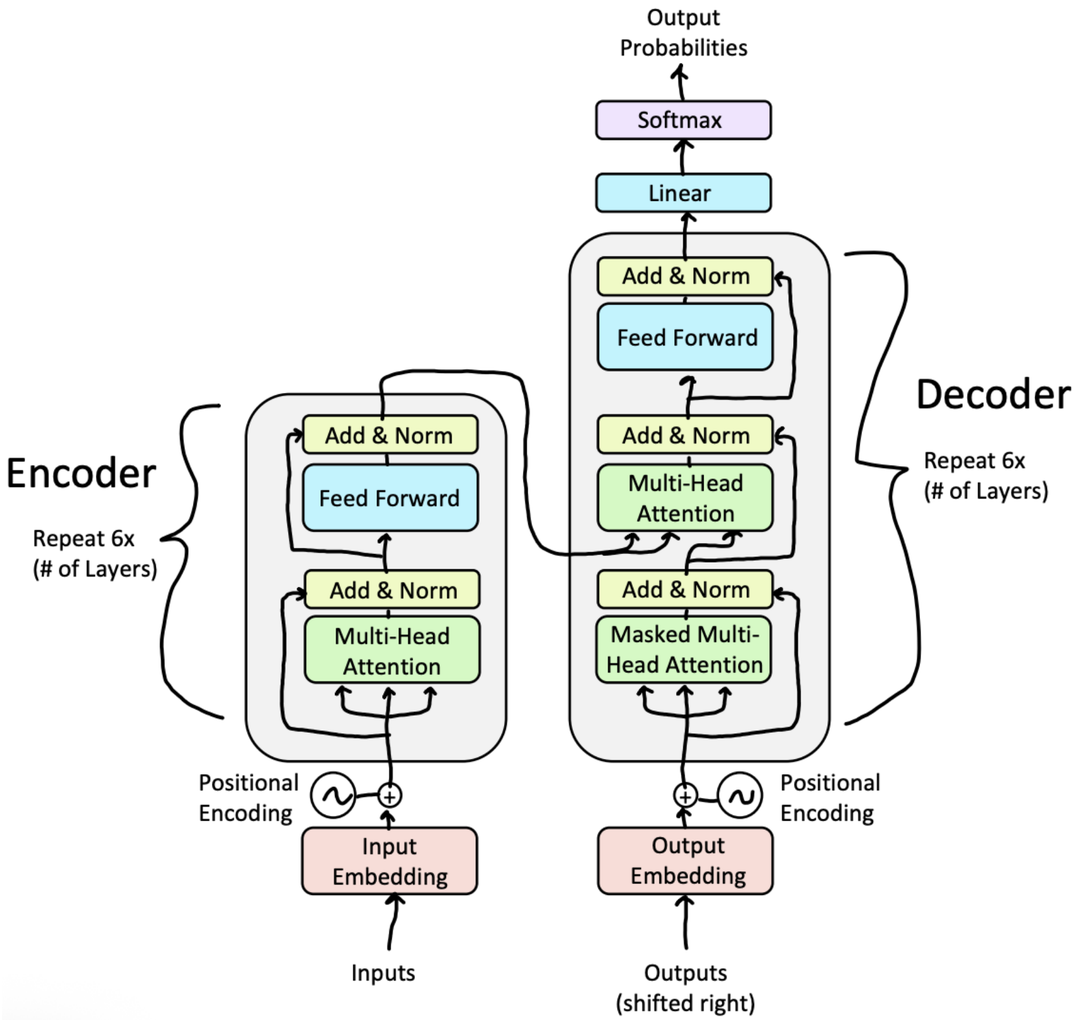
- Add a feedforward layer (w/ residual connection and layer norm)



Decoder: Finishing touches!

- Add a final **linear layer** to project the embeddings into a much longer vector of length vocab size (logits)

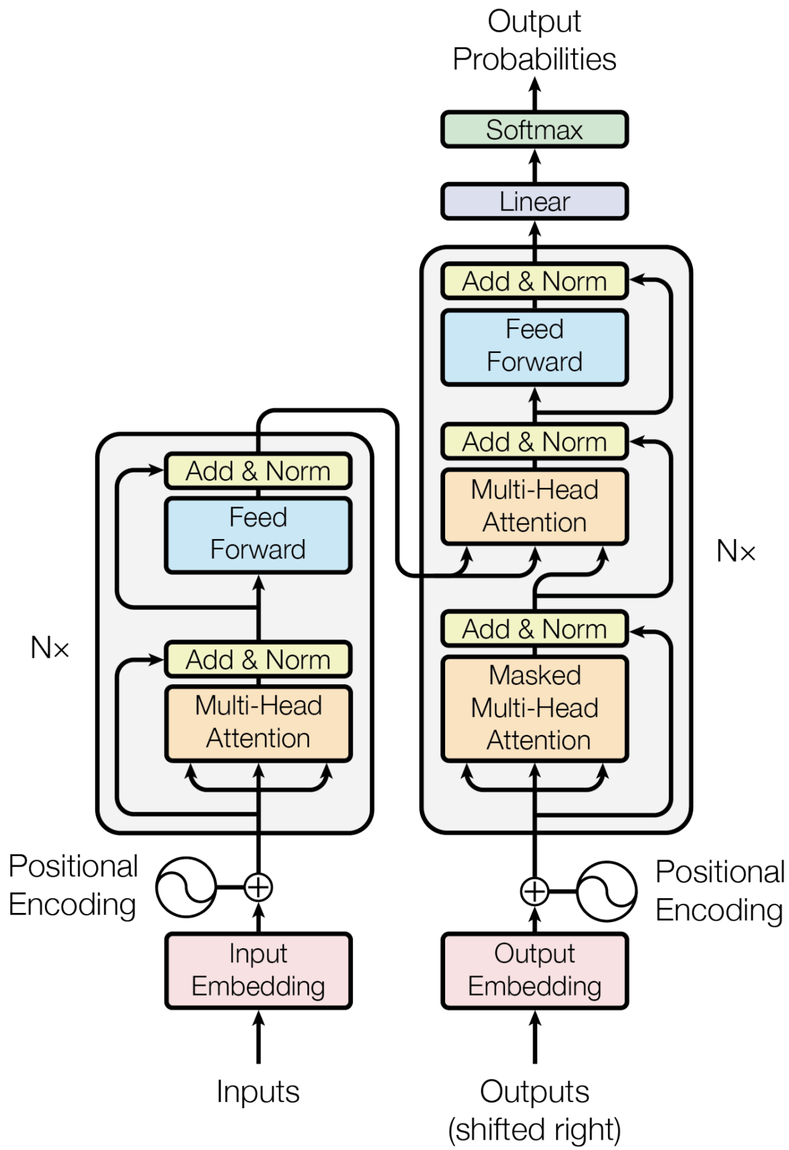
- Add a final **softmax** to generate a probability distribution of possible next words!



Transformers Have Revolutionized NLP

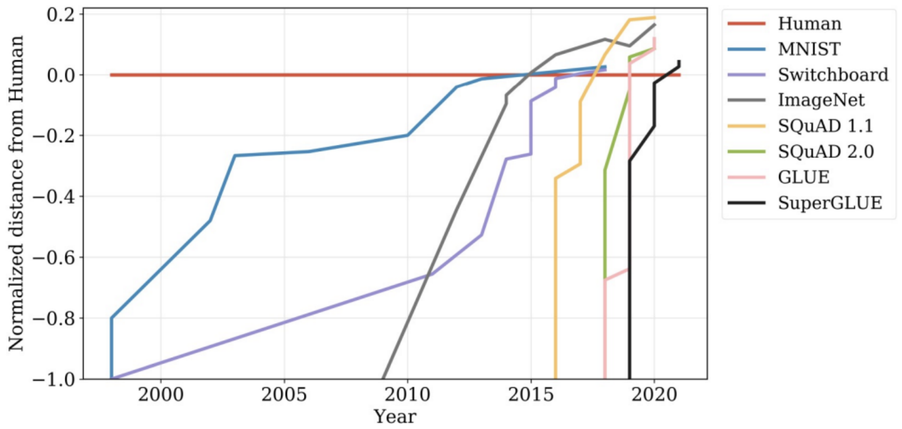


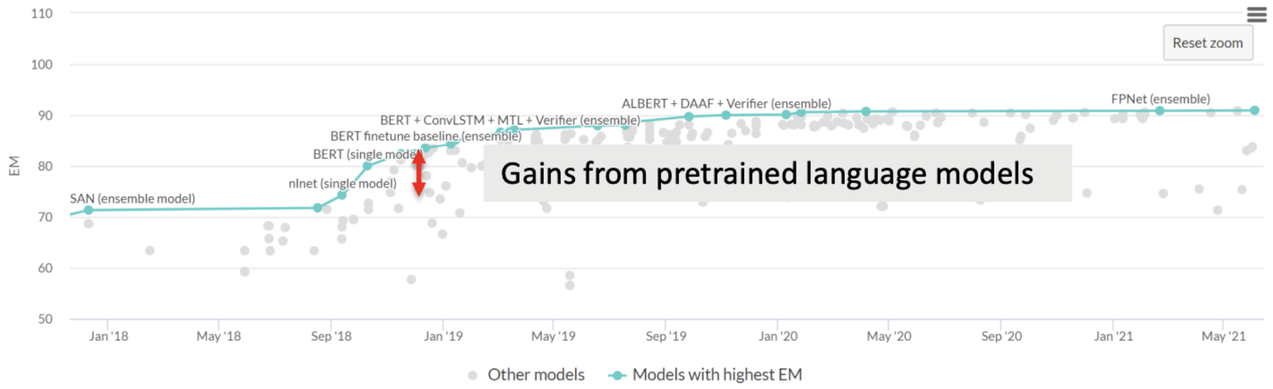
Courtesy of Paramount Pictures



The Pre-training and Fine-tuning Paradigm

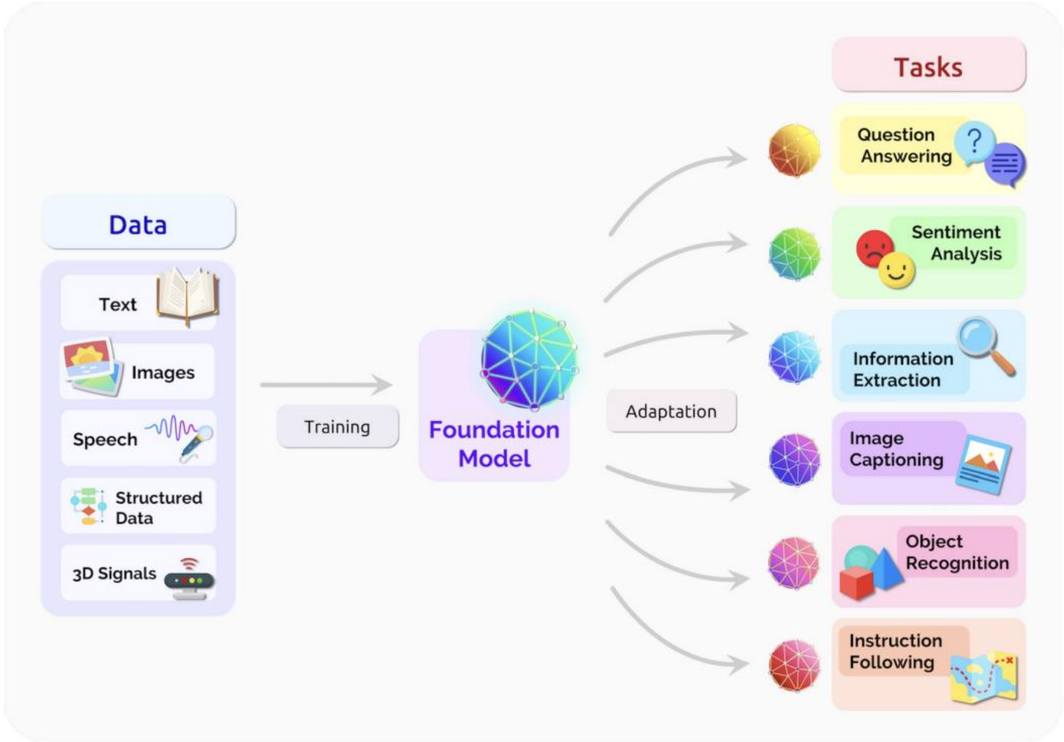
Pretraining Revolution





Pretraining has had a major, tangible impact on how well NLP systems work

Pretraining - Scaling unsupervised learning on the Internet



Key ideas in pretraining

- Make sure your model can process large-scale, diverse datasets
- Don’t use labeled data (otherwise you can’t scale!)
- Compute-aware scaling

Traditional to modern NLP: training paradigm

N-gram language models	→	Neural language models: BERT, GPT
Traditional models: Naive Bayes	→	Neural models: Transformers
Static embeddings: word2vec	→	Contextual embeddings: BERT, GPT
Traditional learning paradigm	→	New learning paradigm: Pretrain, ICL

Question: How to train and use neural language models for different NLP tasks?

Example: Training Transformers for sentiment analysis

Task:



Model:



Transformers

Traditional learning paradigm

Supervised training/fine-tuning only, NO pre-training

- Collect (x, y) task training pairs

Data:

sentence	label
a stirring , funny and finally transporting re imagining of beauty and the beast and 1930s horror films	1
apparently reassembled from the cutting room floor of any given daytime soap	0
they presume their audience won't sit still for a sociology lesson	0
this is a visually stunning rumination on love , memory , history and the war between art and commerce	1
jonathan parker 's bartleby should have been the be all end all of the modern office anomie films	1

Traditional learning paradigm

Supervised training/fine-tuning only, NO pre-training

- Collect (x, y) task training pairs
- Randomly initialize your models f(x) (e.g., vanilla Transformers)

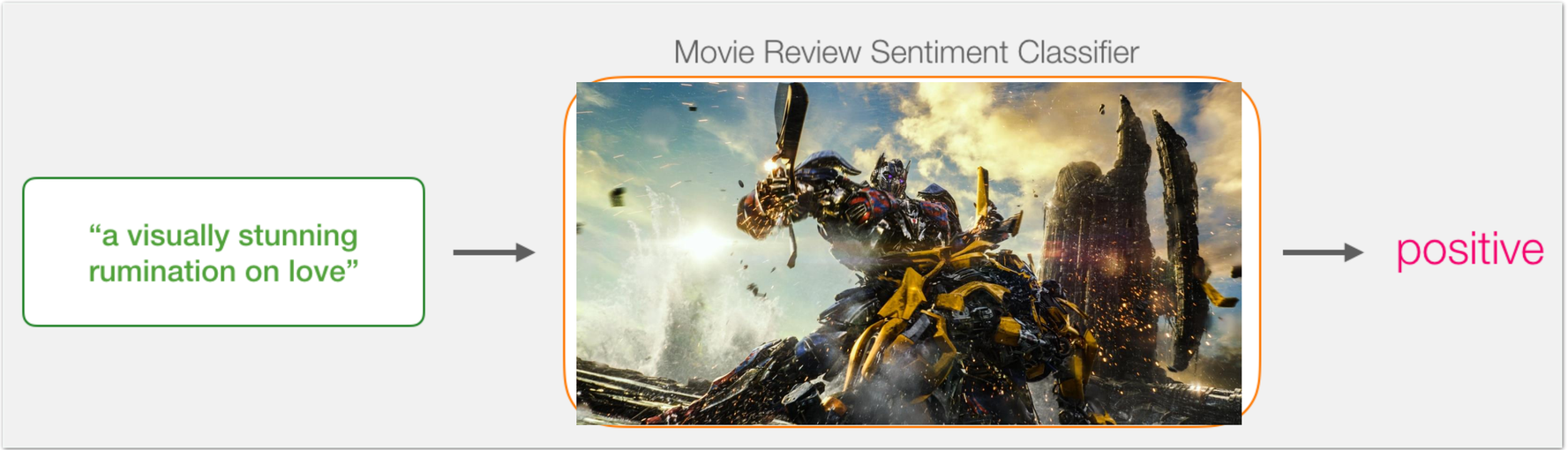


Randomly initialized Transformers
NO pretrained parameters used

Traditional learning paradigm

Supervised training/fine-tuning only, NO pre-training

- Collect (x, y) task training pairs
- Randomly initialize your models f(x) (e.g., vanilla Transformers)
- Train f(x) on (x, y) pairs



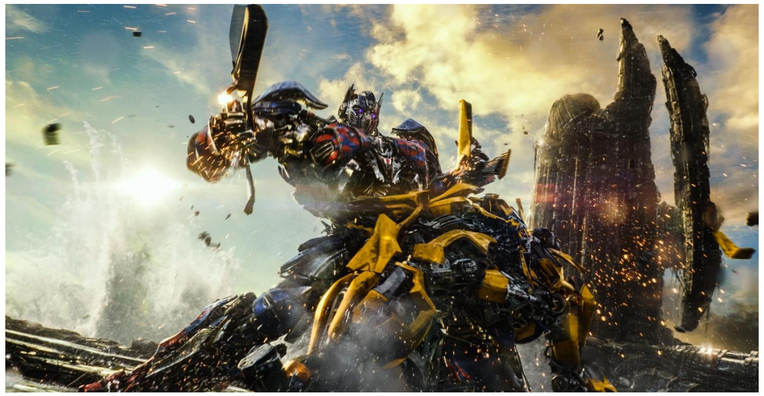


Train Transformers only on task labeled data

Traditional learning paradigm

Supervised training/fine-tuning only, NO pre-training

- Collect (x, y) task training pairs
- Randomly initialize your models f(x) (e.g., vanilla Transformers)
- Train f(x) on (x, y) pairs



Then you get a trained Transformers **ONLY** for sentiment analysis
The model can be: NB, LR, RNNs, LSTM too

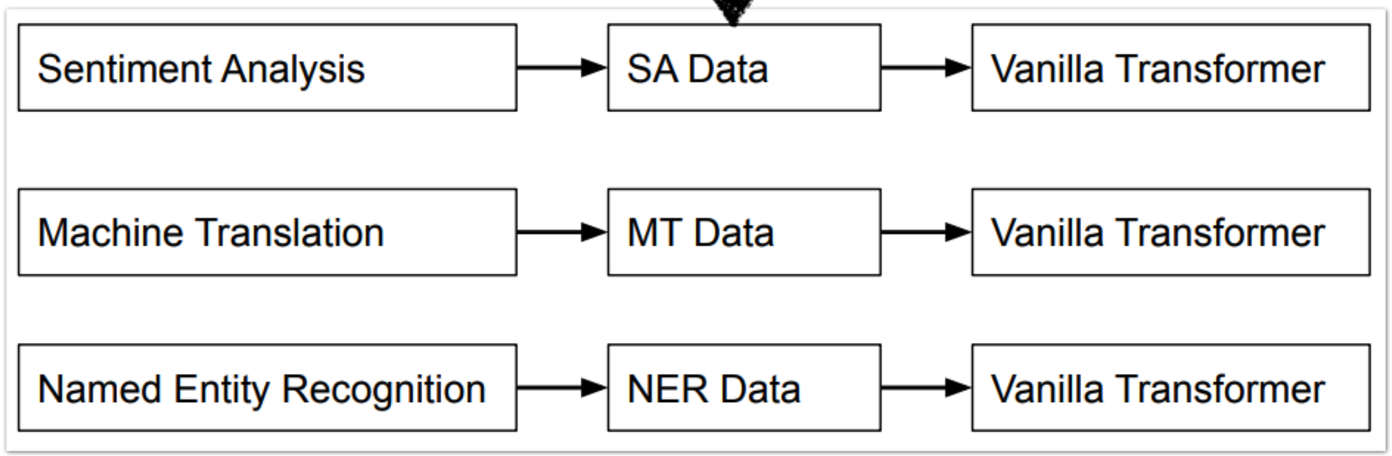
Traditional learning paradigm

Supervised training/fine-tuning only, NO pre-training

Train Transformer or other models separately for each task

sentence	label
a stirring , funny and finally transporting re imagining of beauty and the beast and 1930s horror films	1
apparently reassembled from the cutting room floor of any given daytime soap	0
they presume their audience won't sit still for a sociology lesson	0
this is a visually stunning rumination on love , memory , history and the war between art and commerce	1
jonathan parker 's bartleby should have been the be all end all of the modern office anomie films	1

Supervised fine-tuning only
NO pre-training

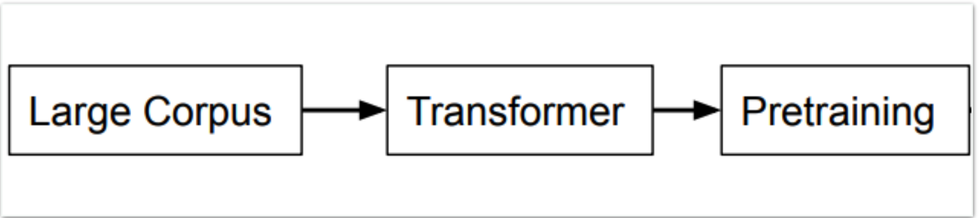


Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.

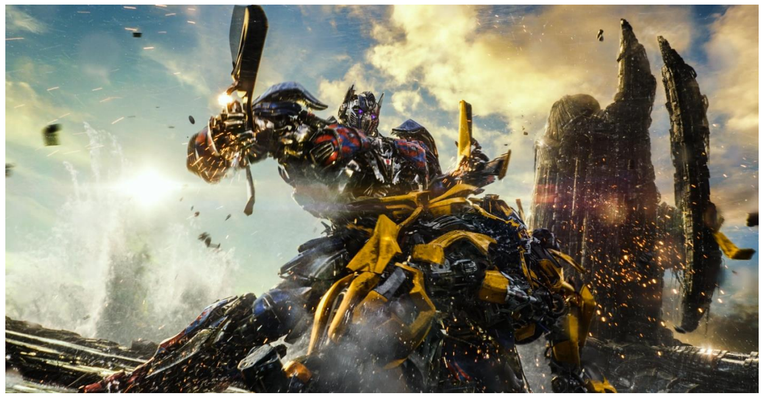




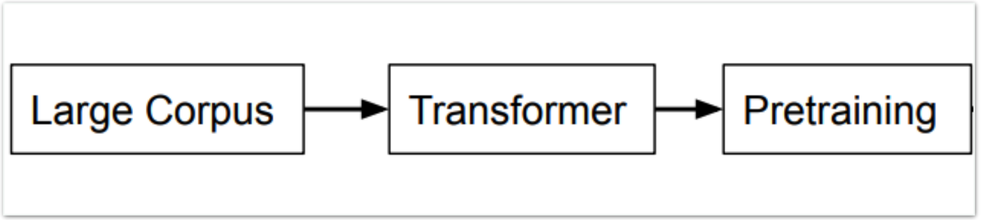
Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.



1. Randomly initialized Transformers



Modern learning paradigm

Pre-training + supervised training/fine-tuning

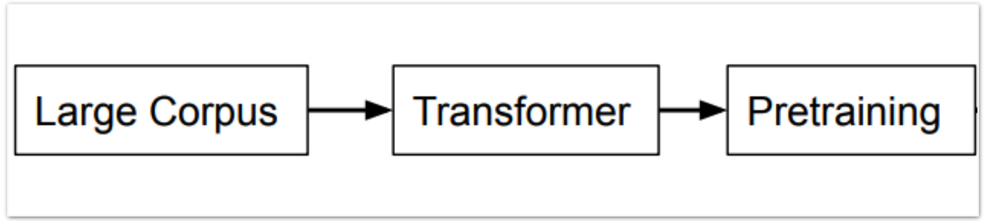
First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.





general training objectives?

2. Pre-train Transformers on large text data with **general objectives**





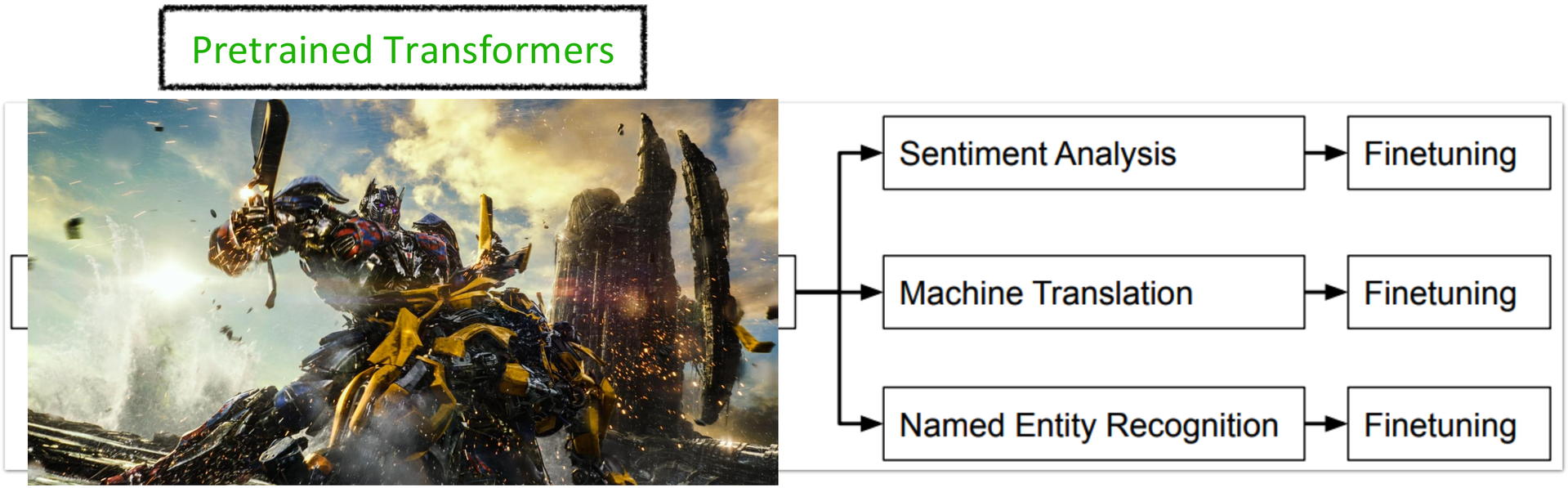
>2M GPU hours

Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.

Then train the pretrained Transformer for a specific task using supervised learning. This is called **finetuning**.

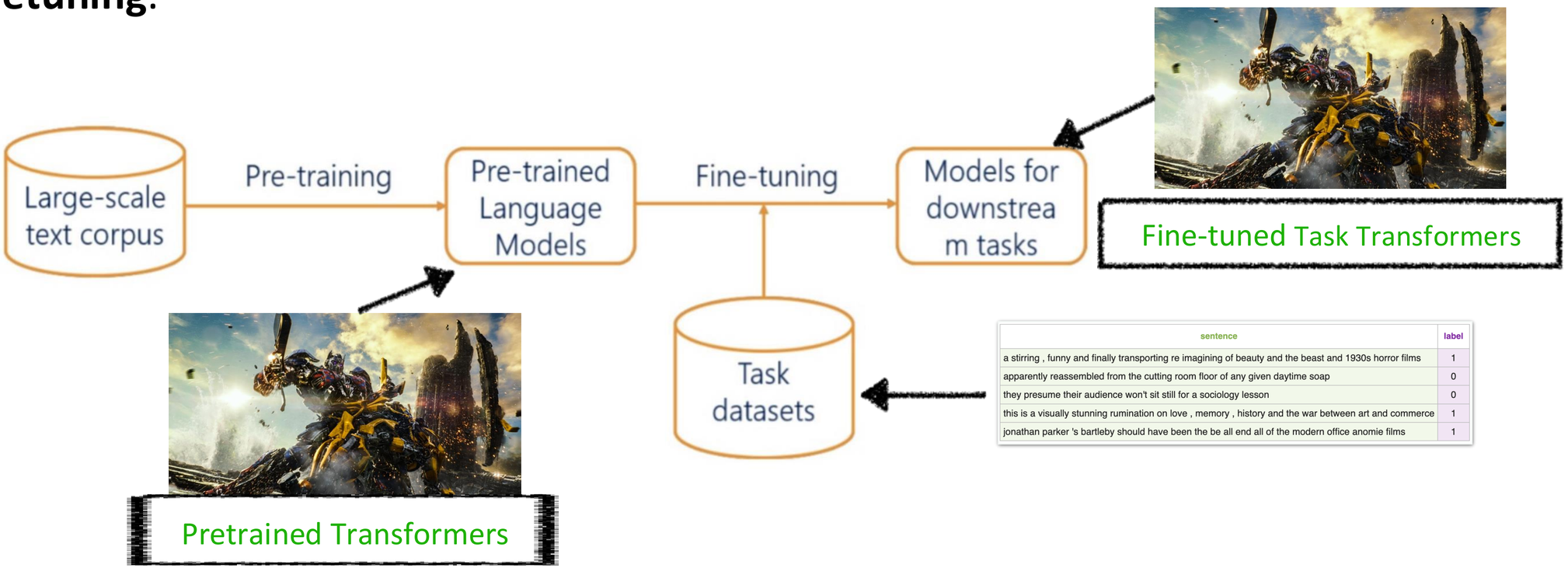


Modern learning paradigm

Pre-training + supervised training/fine-tuning

First train Transformer using a lot of general text using unsupervised learning. This is called **pretraining**.

Then train the pretrained Transformer for a specific task using supervised learning. This is called **finetuning**.



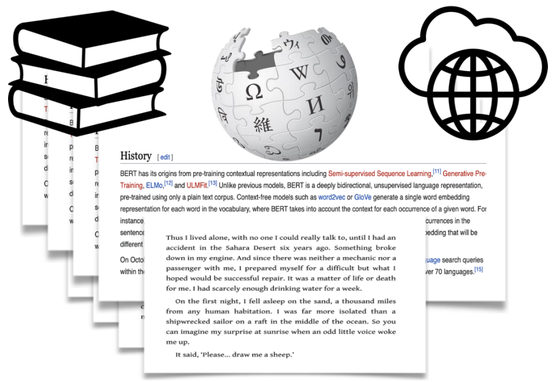
Advantages of pre-training

- **Leveraging rich underlying information** from abundant raw texts.
- **Reducing the reliance of task-specific labeled data** that is difficult or costly to obtain.
- **Initializing model parameters** for more **generalizable** NLP applications.
- **Saving training cost** by providing a reusable model checkpoints.
- **Providing robust representation** of language contexts.

Pretraining: training objectives?

During pretraining, we have a large text corpus (**no task labels**)

- **Key question: what labels or objectives used to train the vanilla Transformers?**



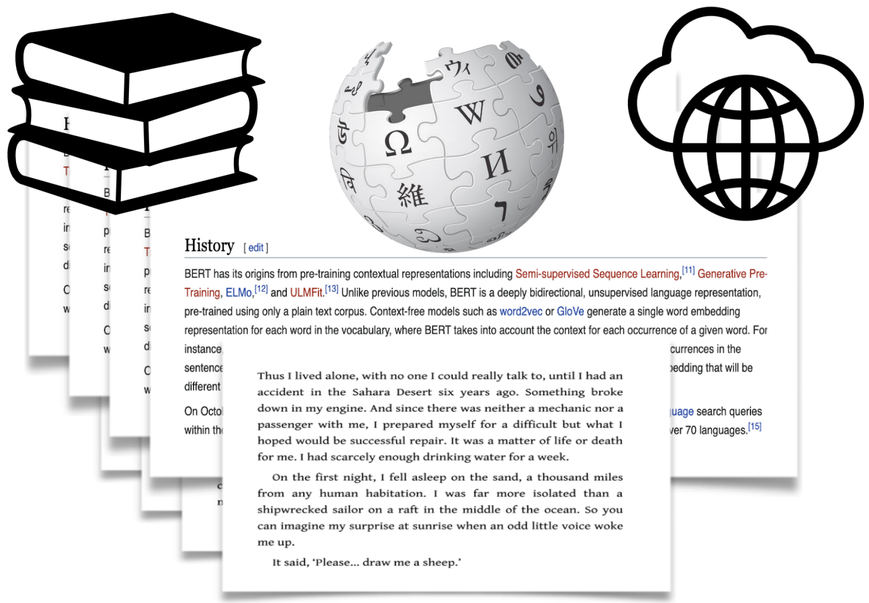


Training labels/objectives?

Pretraining: training objectives?

During pretraining, we have a large text corpus (**no task labels**)

- **Key question: what labels or objectives used to train the vanilla Transformers?**



Most of pre-training is reconstructing the input

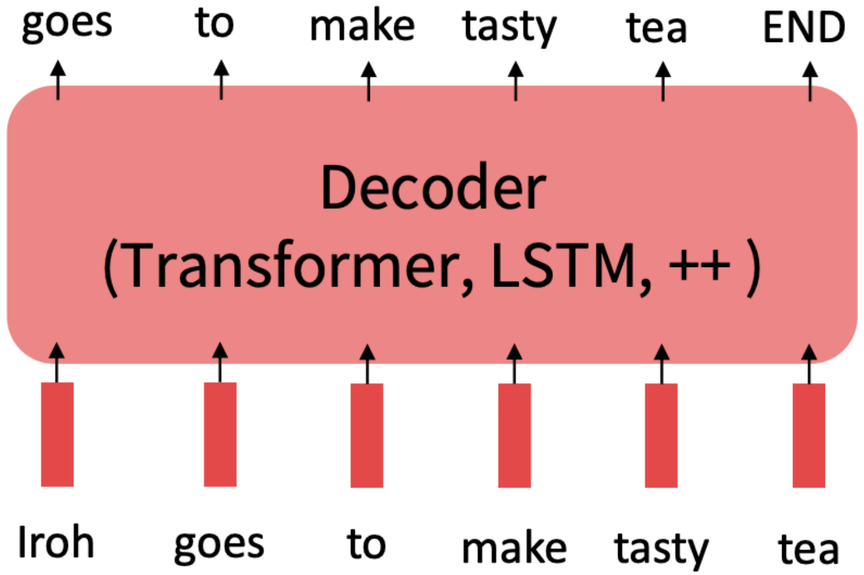
University of Southern California is located in _____ .

What can we learn from reconstructing the input?

- I went to the ocean and saw fish, turtles, _____ and seals.
  - General semantics
- I put ____ fork down on the table
  - Syntactic constraints
- The woman walked across the street checking for traffic over _____ shoulder.
  - Co-reference, relations between different entities within the sentence
- Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was _____.
  - Sentiment

Pretraining: Language Models

- Recall the language modeling task:
  - Model $p_\theta(w_t | w_{1:t-1})$, the probability distribution over words given their past contexts.
  - There's lots of data for this! (In English.)
- Pretraining through language modeling:
  - Train a neural network to perform language modeling on a large amount of text.
  - Save the network parameters.



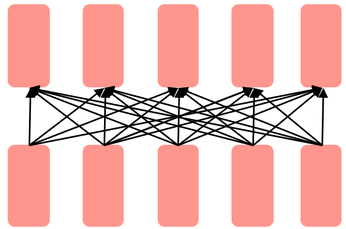
Semi-supervised Sequence Learning

Andrew M. Dai
Google Inc.
adai@google.com

Quoc V. Le
Google Inc.
qvl@google.com

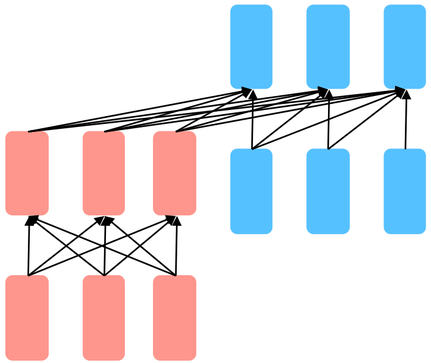
Pre-training architectures

Encoder



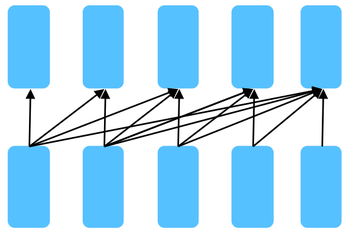
- Bidirectional; can condition on the future context

Encoder-Decoder



- Map two sequences of different length together

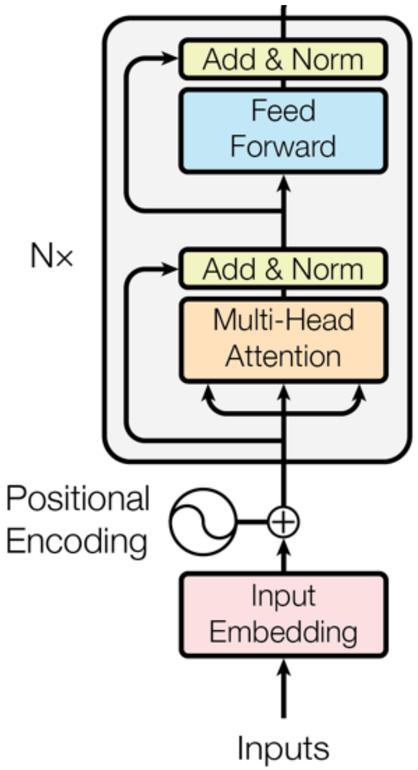
Decoder



- Language modeling; can only condition on the past context

BERT: Bidirectional Encoder Representations from Transformers

(Released in 2018/10)



- It is a fine-tuning approach based on a deep **bidirectional Transformer encoder** instead of a Transformer decoder
- The key: learn representations based on **bidirectional contexts**

  Example #1: we went to the river bank.

  Example #2: I need to go to bank to make a deposit.

- Two new pre-training objectives:
  - **Masked language modeling (MLM)**
  - Next sentence prediction (NSP) - Later work shows that NSP hurts performance though..



(Devlin et al, 2019): BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Masked Language Modeling (MLM)

- Q: Why we can't do language modeling with bidirectional models?



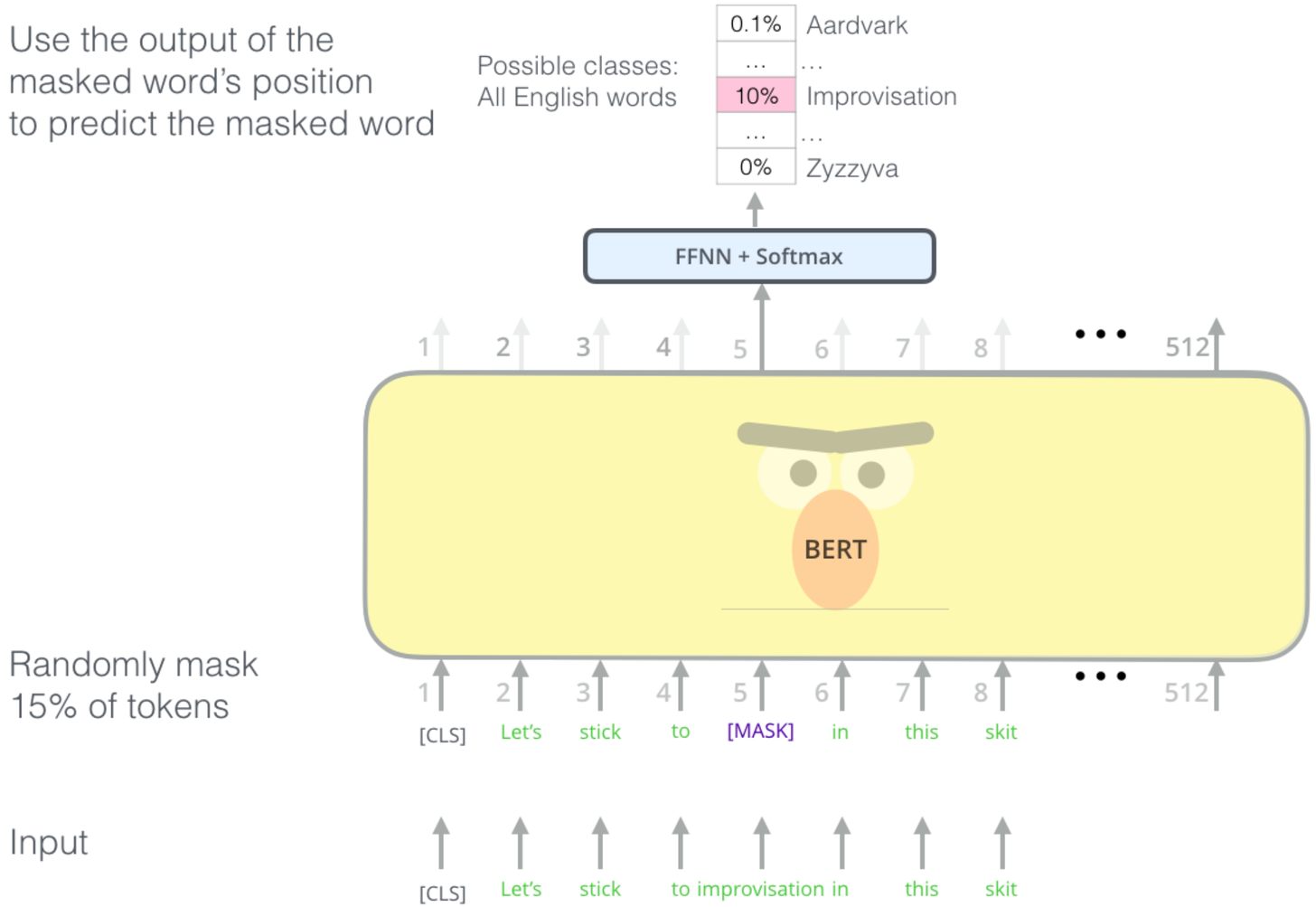
- Solution: Mask out k% of the input words, and then predict the masked words

store gallon

the man went to [MASK] to buy a [MASK] of milk

k = 15% in practice

Masked Language Modeling (MLM)



http://jalammar.github.io/illustrated-bert/

MLM: 80-10-10 corruption

For the 15% predicted words,

- 80% of the time, they replace it with [MASK] token

went to the store ⟶ went to the [MASK]

- 10% of the time, they replace it with a random word in the vocabulary

went to the store ⟶ went to the running

- 10% of the time, they keep it unchanged

went to the store ⟶ went to the store

Why? Because [MASK] tokens are never seen during fine-tuning

(See Table 8 of the paper for an ablation study)

Next Sentence Prediction (NSP)

- Motivation: many NLP downstream tasks require understanding the relationship between two sentences (natural language inference, paraphrase detection, QA)
- NSP is designed to reduce the gap between pre-training and fine-tuning

[CLS]: a special token always at the beginning

[SEP]: a special token used to separate two segments

Input = [CLS] the man went to [MASK] store [SEP]

he bought a gallon [MASK] milk [SEP]

Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]

penguin [MASK] are flight ##less birds [SEP]

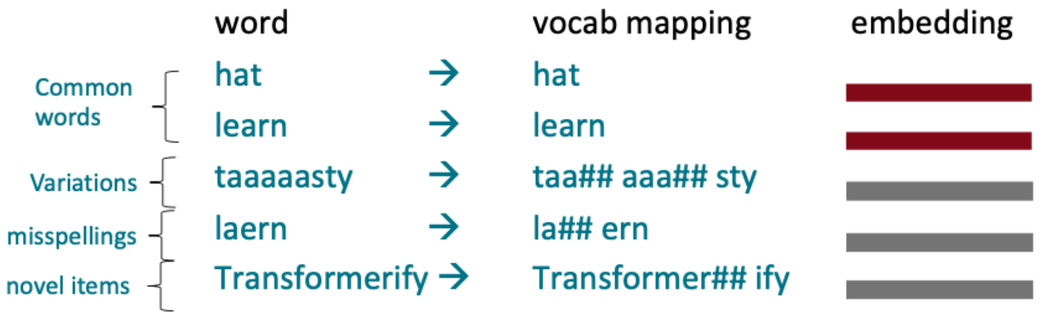
Label = NotNext

They sample two contiguous segments for 50% of the time and another random segment from the corpus for 50% of the time

This actually hurts model learning based on later work!

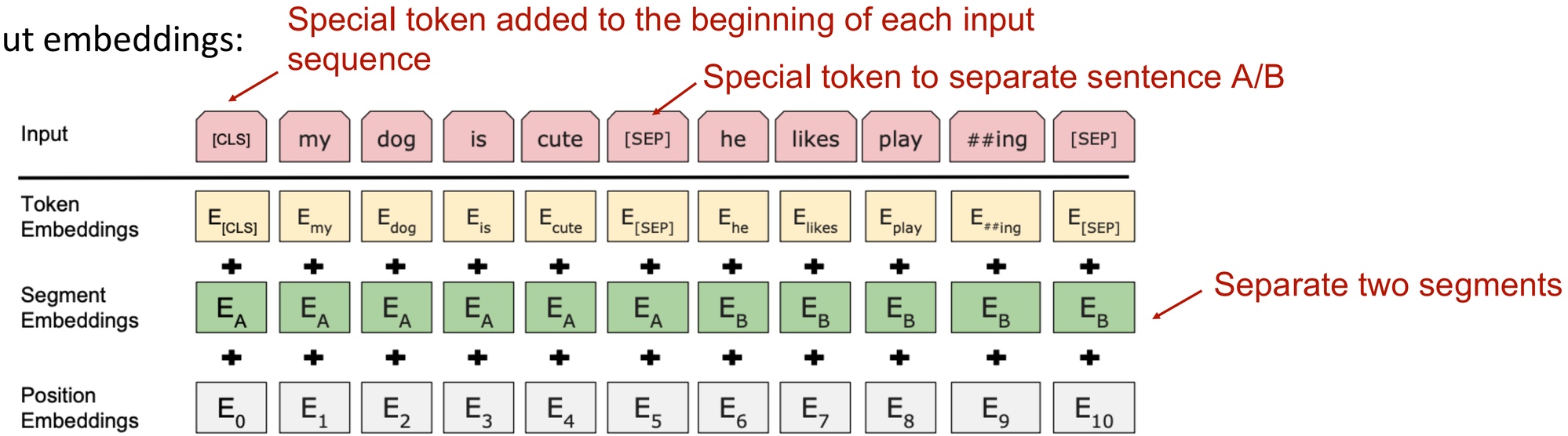
BERT pre-training

- Vocabulary size: 30,000 wordpieces (common sub-word units) (Wu et al., 2016)

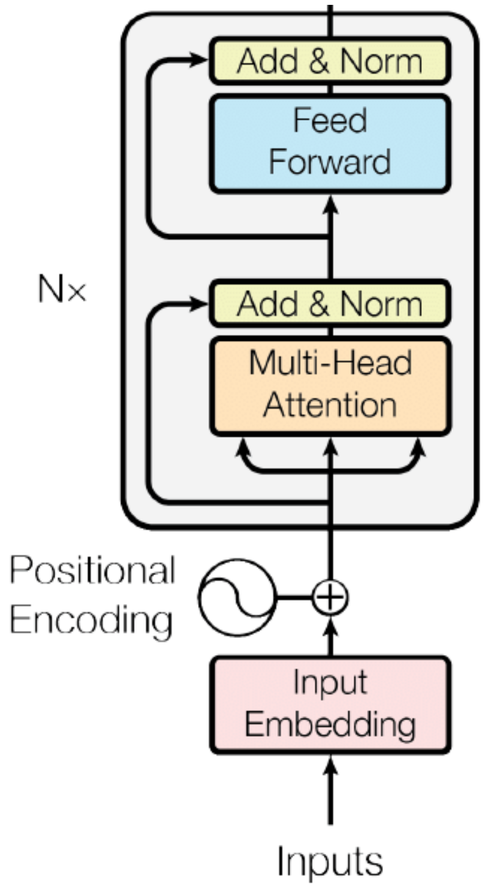


(Image: Stanford CS224N)

- Input embeddings:



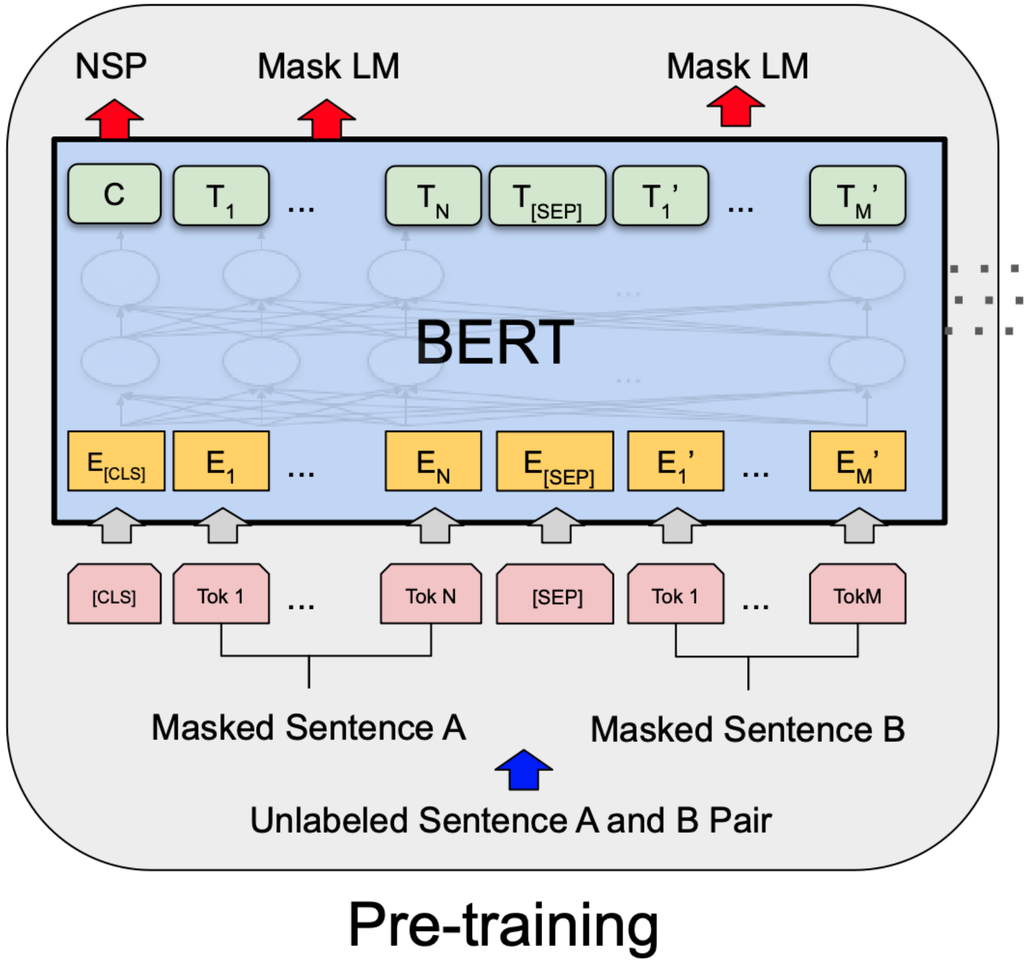
BERT pre-training



- BERT-base: 12 layers, 768 hidden size, 12 attention heads, 110M parameters
- BERT-large: 24 layers, 1024 hidden size, 16 attention heads, 340M parameters

- Training corpus: Wikipedia (2.5B) + BooksCorpus (0.8B)
- Max sequence size: 512 wordpiece tokens (roughly 256 and 256 for two non-contiguous sequences)
- Trained for 1M steps, batch size 128k

BERT pre-training

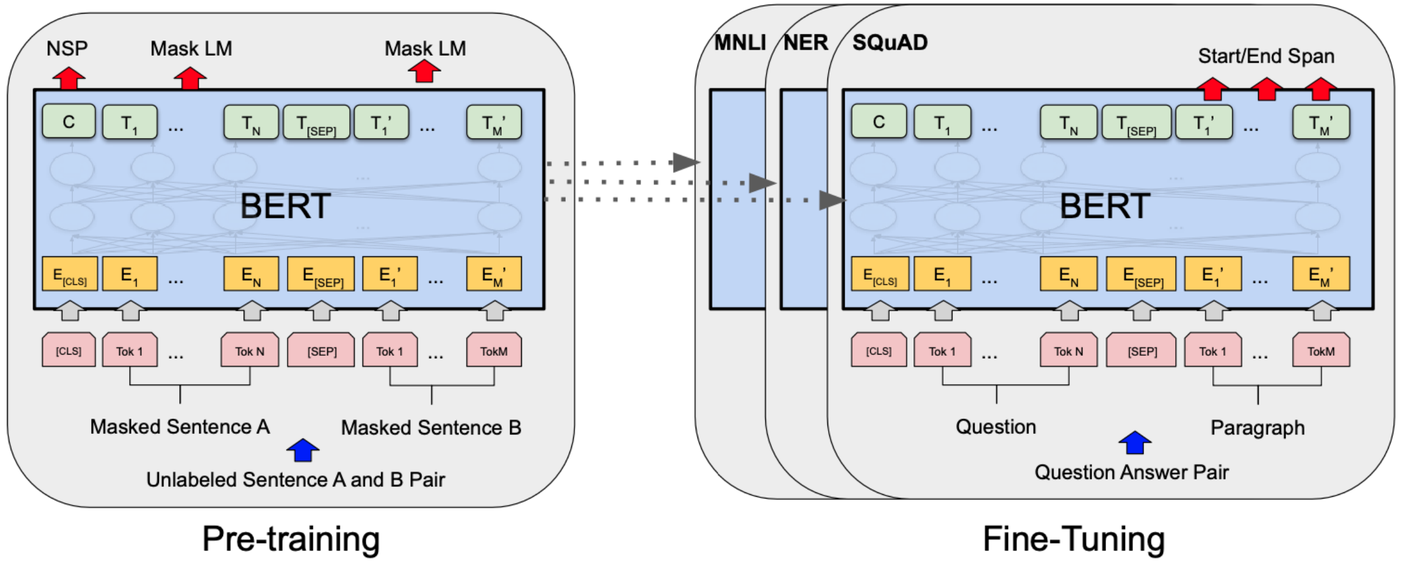


Pre-training

- MLM and NSP are trained together
- [CLS] is pre-trained for NSP
- Other token representations are trained for MLM

Pretraining / fine-tuning

“Pre-train” a model on a large dataset for task X, then “fine-tune” it on a dataset for task Y



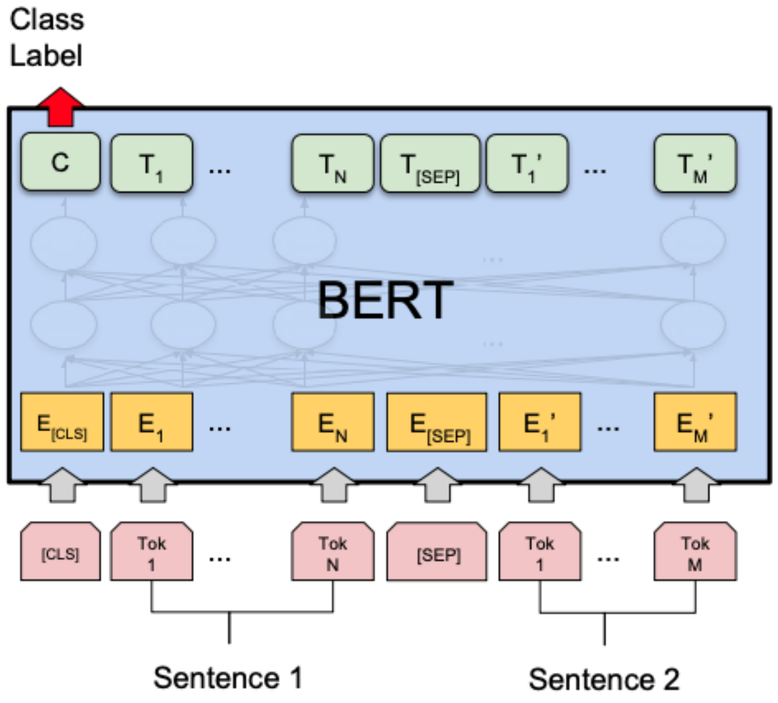
“**Fine-tuning** is the process of **taking the network learned by these pre-trained models**, and **further training the model**, often via an added neural net classifier that takes the top layer of the network as input, to perform some downstream task.”

Fine-tuning is a training process and takes **gradient descent steps**!

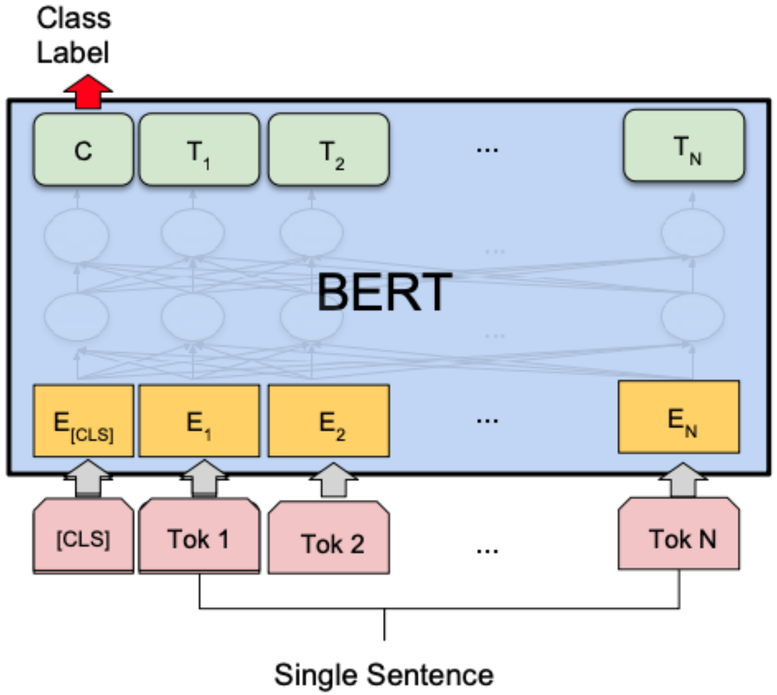
BERT fine-tuning

"Pretrain once, finetune many times."

sentence-level tasks



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



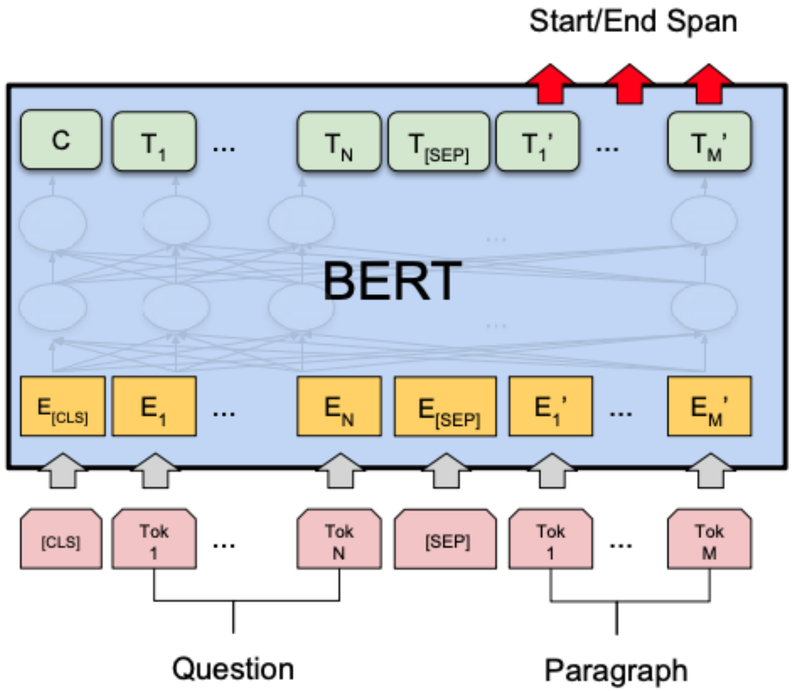
(b) Single Sentence Classification Tasks: SST-2, CoLA

- **QQP:** Quora Question Pairs (detect paraphrase questions)
- **QNLI:** natural language inference over question answering data
- **SST-2:** sentiment analysis

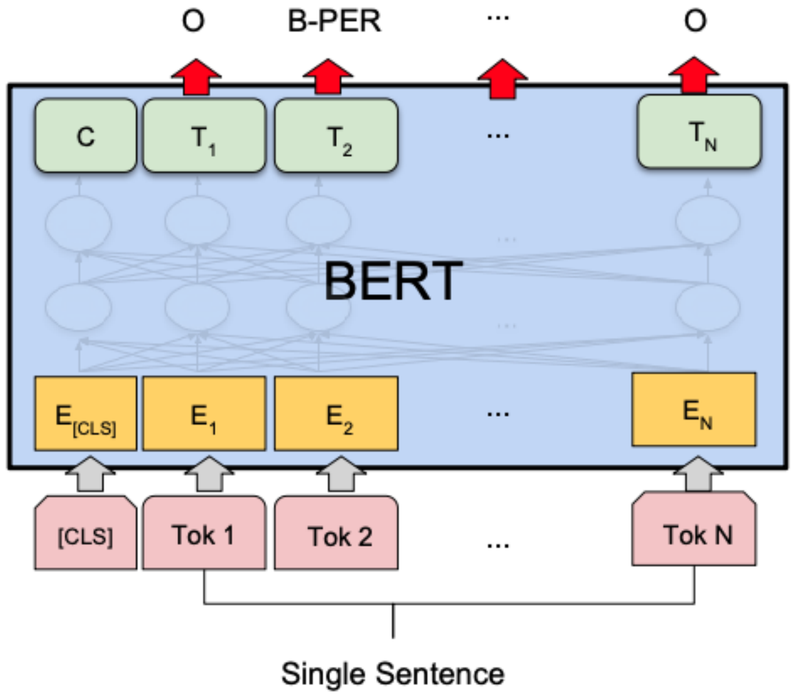
BERT fine-tuning

"Pretrain once, finetune many times."

token-level tasks



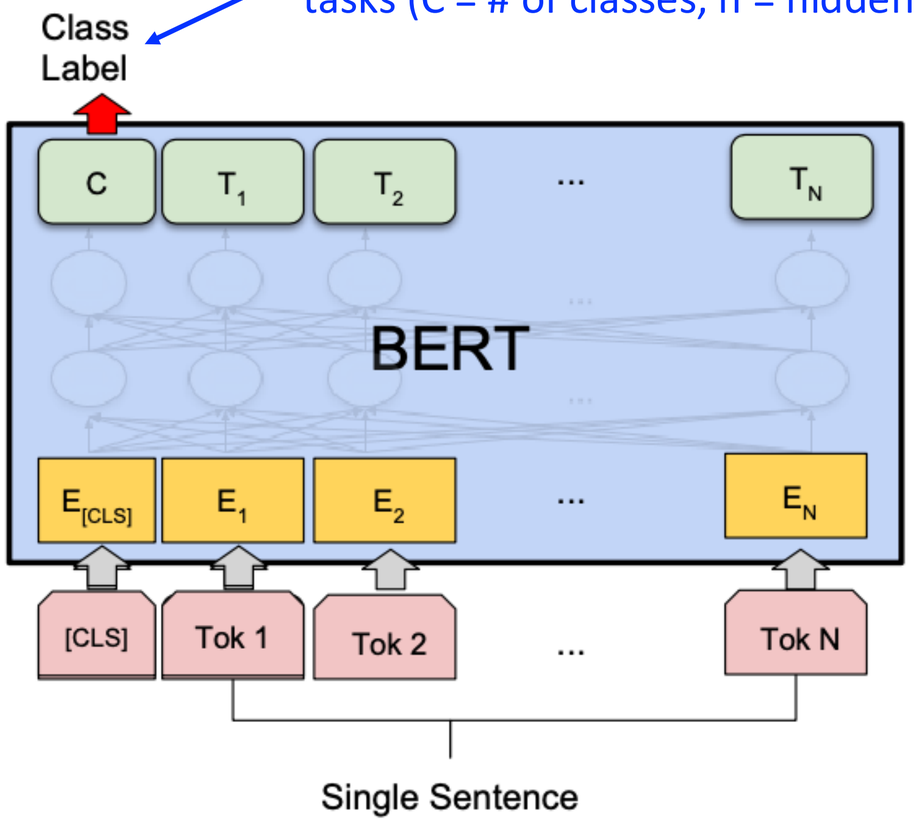
(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Example: sentiment classification

We just need to introduce $C \times h$ parameters for classification tasks (C = # of classes, h = hidden size)!



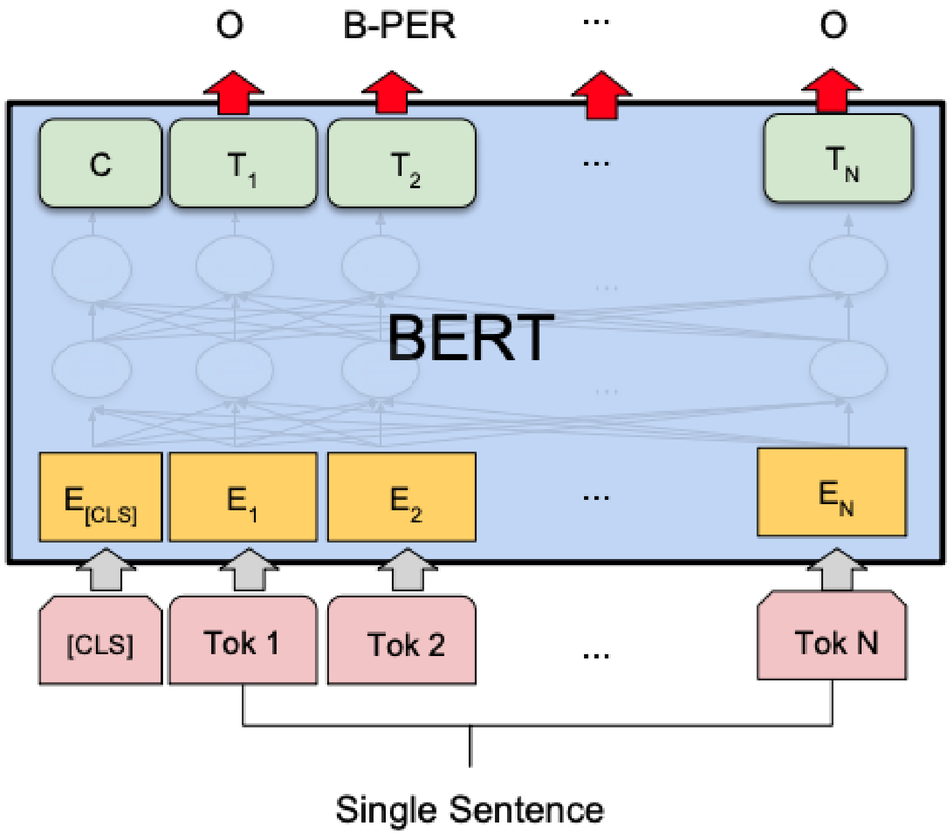
$$P(y = k) = softmax_k(\mathbf{W}_o \mathbf{h}_{[CLS]})$$

$$\mathbf{W}_o \in \mathbb{R}^{C \times h}$$

All the parameters will be learned together (original BERT parameters + new classifier parameters)

Example: named entity recognition (NER)

We just need to introduce $C \times h$ parameters for classification tasks (C = # of classes, h = hidden size)!



$$P(y_i = k) = softmax_k(\mathbf{W}_o \mathbf{h}_i)$$

$$\mathbf{W}_o \in \mathbb{R}^{C \times h}$$

Experimental results: GLUE

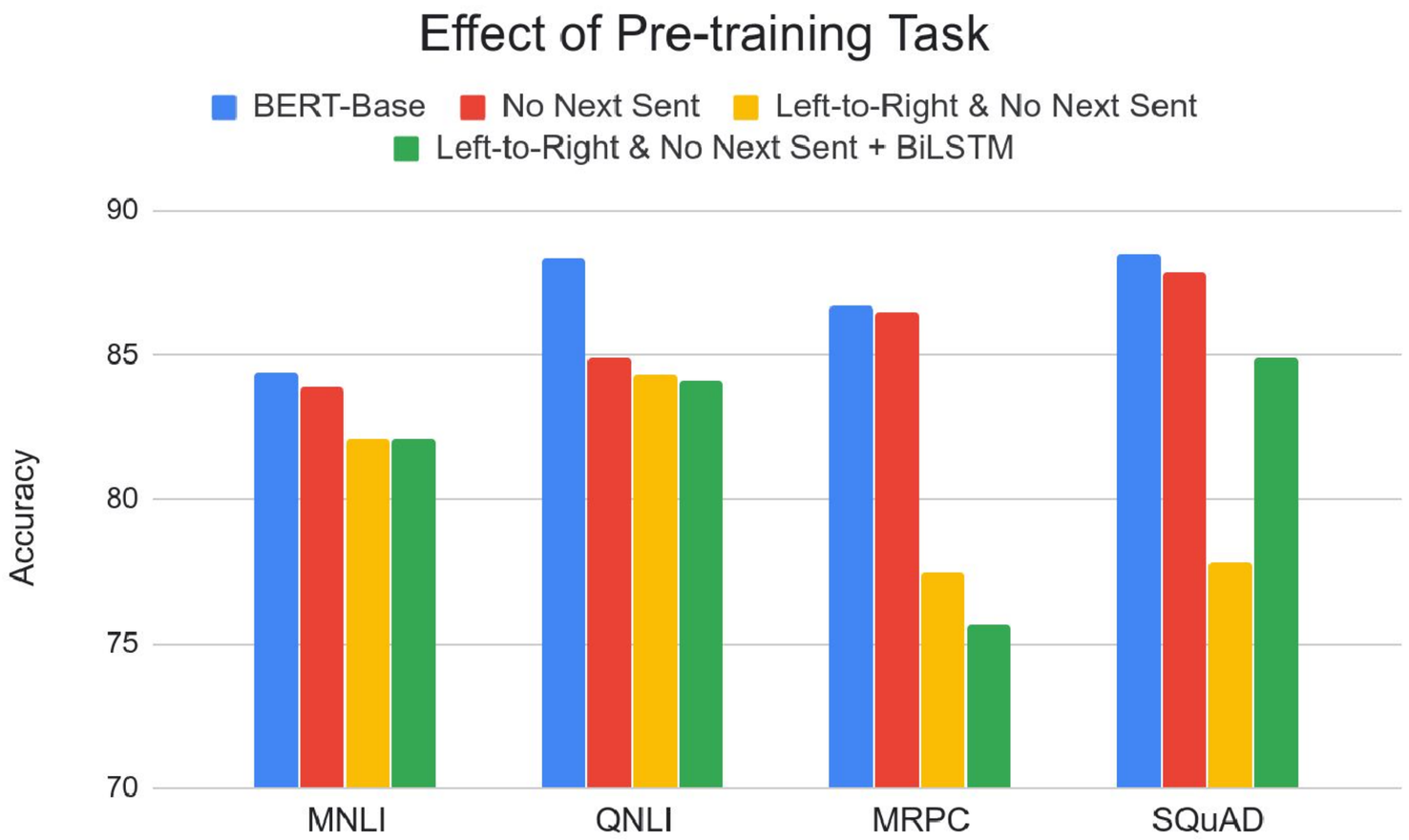
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Experimental results: SQuAD

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
$\text{BERT}_{\text{BASE}}$ (Single)	80.8	88.5	-	-
$\text{BERT}_{\text{LARGE}}$ (Single)	84.1	90.9	-	-
$\text{BERT}_{\text{LARGE}}$ (Ensemble)	85.8	91.8	-	-
$\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA)	**84.2**	**91.1**	**85.1**	**91.8**
$\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA)	**86.2**	**92.2**	**87.4**	**93.2**

SQuAD = Stanford Question Answering dataset

Ablation study: pre-training tasks



- MLM >> left-to-right LMs
- NSP improves on some tasks
- Note: later work (Joshi et al., 2020; Liu et al., 2019) argued that NSP is not useful

Ablation study: model sizes

layers ↓ hidden size ↓ # of heads ↙

Hyperparams			Dev Set Accuracy			
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

The bigger, the better!

Encoder: other variations of BERT

- **ALBERT [Lan et al., 2020]**: incorporates two parameter reduction techniques that lift the major obstacles in scaling pre-trained models
- **DeBERTa [He et al., 2021]:** decoding-enhanced BERT with disentangled attention
- **SpanBERT [Joshi et al., 2019]:** masking contiguous spans of words makes a harder, more useful pre-training task
- **ELECTRA [Clark et al., 2020]:** corrupts texts by replacing some tokens with plausible alternatives sampled from a small generator network, then train a discriminative model that predicts whether each token in the corrupted input was replaced by a generator sample or not.
- **DistilBERT [Sanh et al., 2019]:** distilled version of BERT that's 40% smaller
- **TinyBERT [Jiao et al., 2019]:** distill BERT for both pre-training & fine-tuning
- **…**

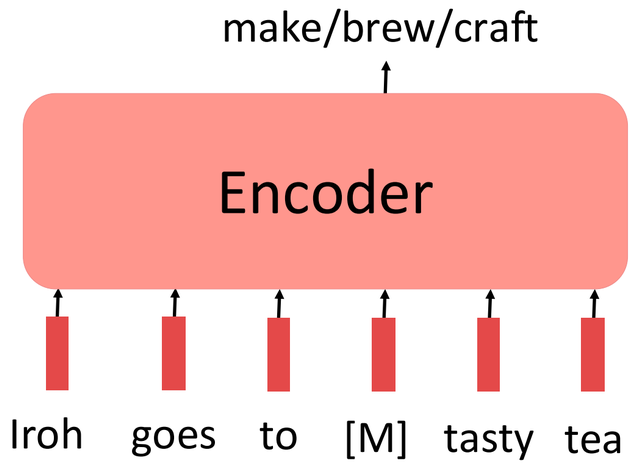
Encoder: pros & cons

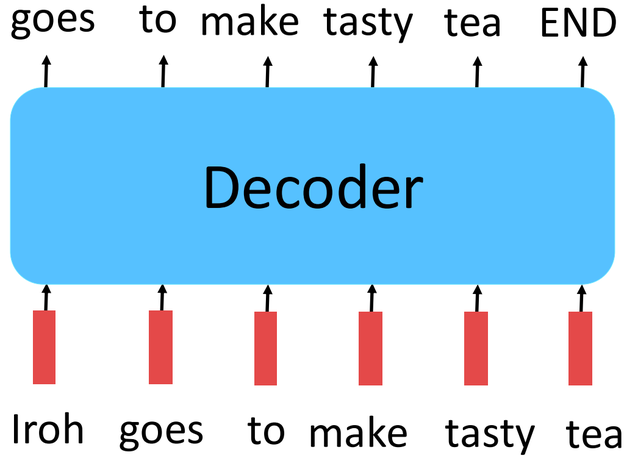


- Consider both left and right context
- Capture intricate contextual relationships



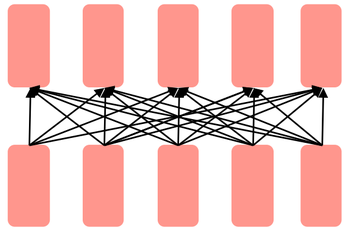
- Not good at generating open-text from left-to-right, one token at a time





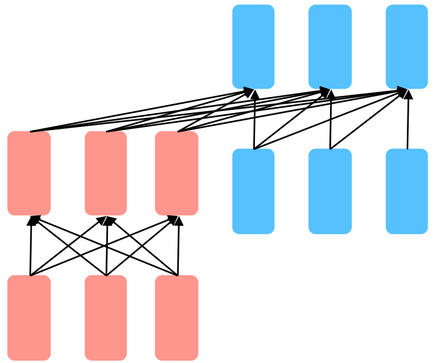
Pre-training architectures

Encoder



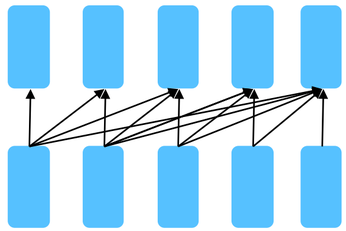
- Bidirectional; can condition on the future context

Encoder-Decoder



- Map two sequences of different length together

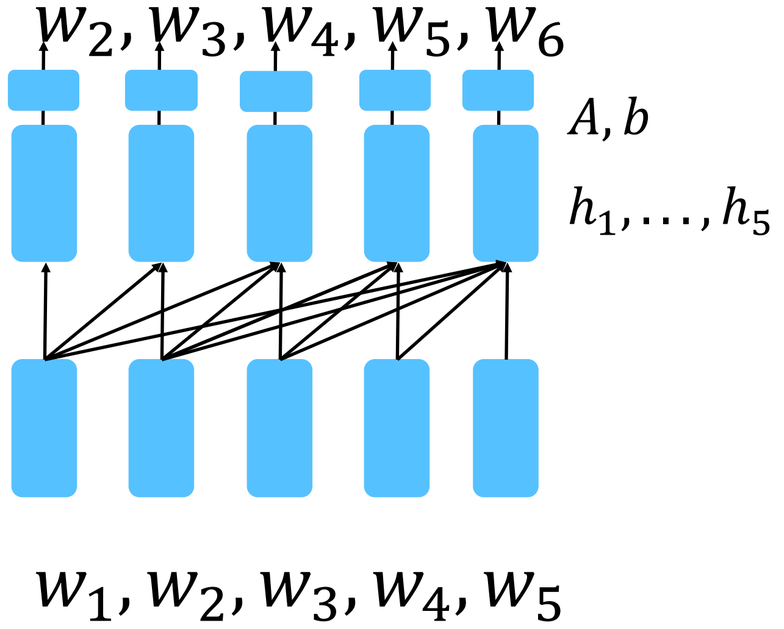
Decoder



- Language modeling; can only condition on the past context

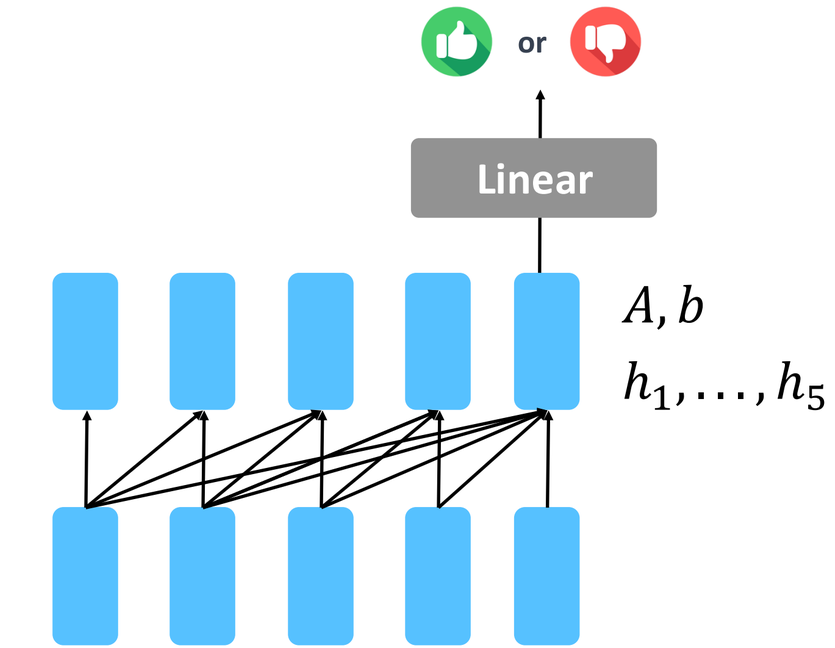
Decoder: training objective

- Many most famous generative LLMs are **decoder-only**
  - e.g., GPT1-5, Llama1-3
- **Language modeling!** Natural to be used for **open-text generation**
- **Conditional LM:** $p(w_t|w_1, \ldots, w_{t-1}, x)$
  - Conditioned on a source context x to generate from left-to-right
- Can be fine-tuned for **natural language generation (NLG)** tasks, e.g., dialogue, summarization.



Decoder: training objective

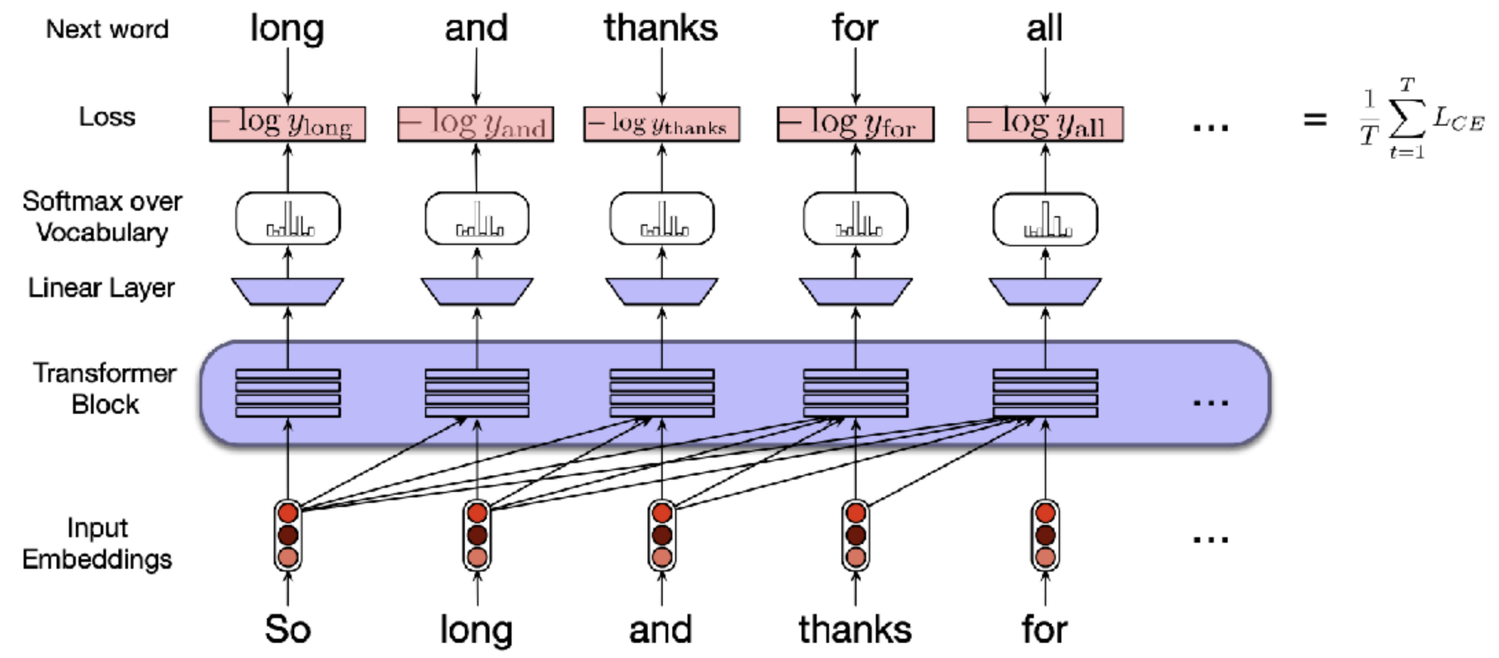
- Customizing the pre-trained model for downstream tasks:
  - Add a **linear layer** on top of the last hidden layer to make it a classifier!
  - During fine-tuning, trained the randomly **initialized linear layer**, along with **all parameters** in the neural net.



Is Santa Claus real figure?

Generative Pre-Training (GPT)

- Use a **Transformer decoder** (unidirectional; left-to-right) instead of LSTMs
- Use **language modeling** as a pre-training objective
- Trained on longer segments of text (**512 BPE tokens**), not just single sentences



(Radford et al, 2018): Improving Language Understanding by Generative Pre-Training

Decoder: GPT

Generative **P**re-trained **T**ransformer

Radford et al., 2018

